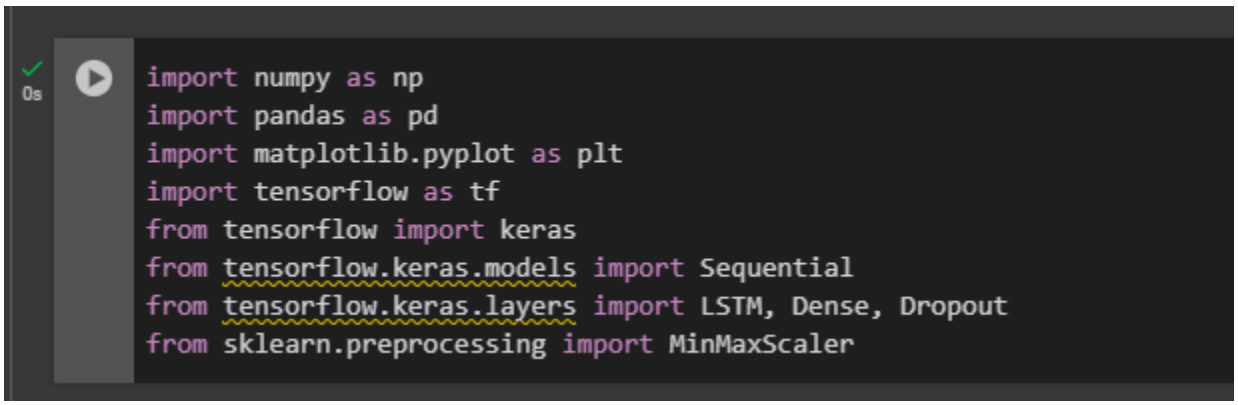
Time Series Data Project

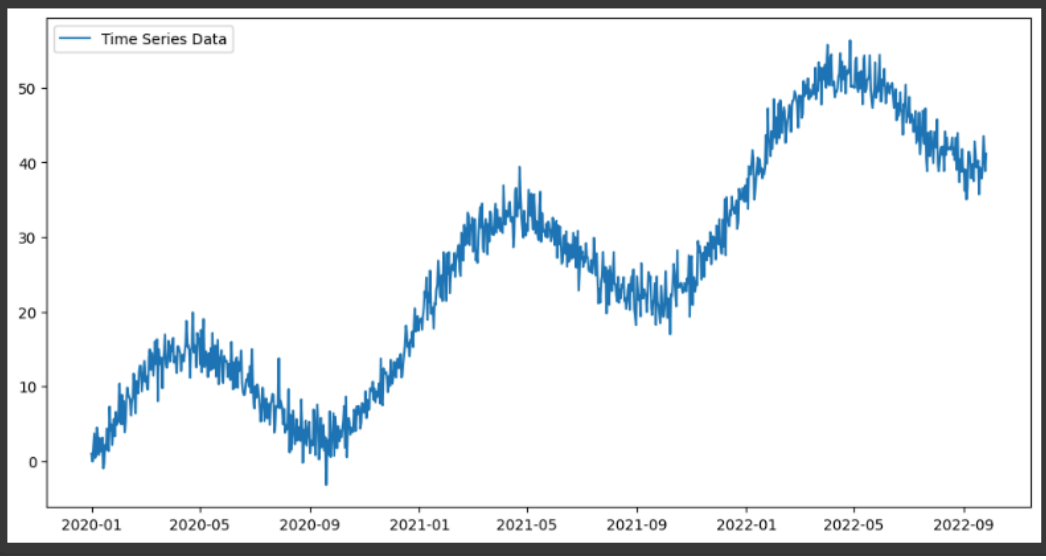
IMPORT LIBRARIES



The screenshot above shows the snippet of the code where libraries are imported. These libraries are needed to build and train the model that will be made for forecasting. For example, “matplotlib.pyplot as plt” allows us to create visualizations to help us better understand the data that will be used and processed.

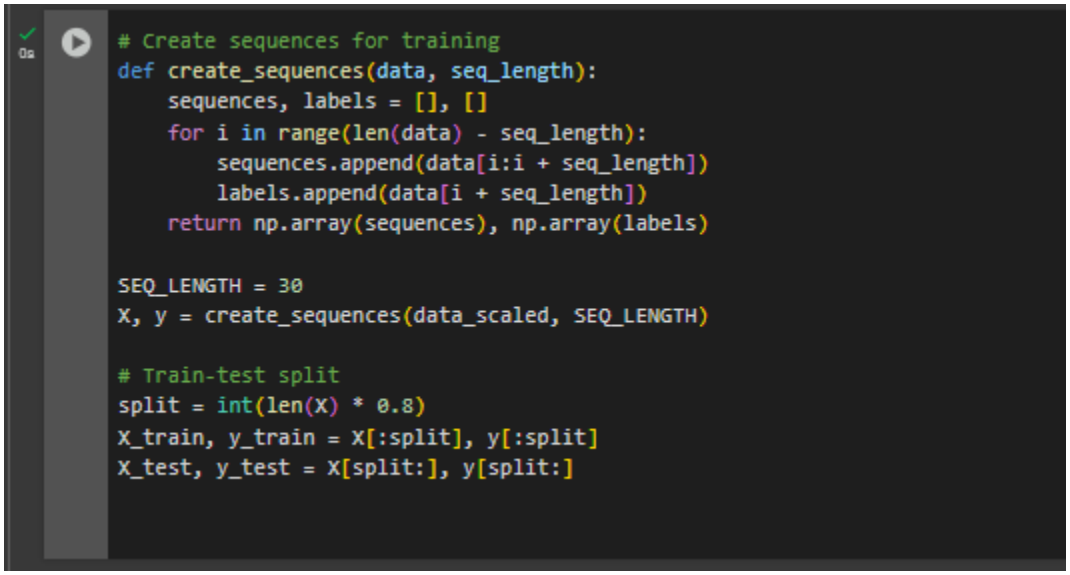
TIME SERIES DATA

```python
def generate_synthetic_data(n=1000):
    np.random.seed(42)
    time = np.arange(n)
    seasonal_pattern = 10 * np.sin(2 * np.pi * time / 365)  # Yearly seasonality
    trend = 0.05 * time  # Linear trend
    noise = np.random.normal(0, 2, n)  # Gaussian noise
    series = trend + seasonal_pattern + noise
    return pd.DataFrame({'Date': pd.date_range(start='2020-01-01', periods=n, freq='D'), 'Value': series})

# Load dataset
df = generate_synthetic_data()
df.set_index('Date', inplace=True)
plt.figure(figsize=(12, 6))
plt.plot(df, label='Time Series Data')
plt.legend()
plt.show()

# Normalize data
scaler = MinMaxScaler(feature_range=(0, 1))
data_scaled = scaler.fit_transform(df)
```



The code snippet shown generates synthetic time series data with a trend, seasonality, and noise. This will be processed and be visualized which can be shown with the graph above. It shows us a yearly seasonal pattern where we can see that in different years there's an increase and decrease of the trend from the dataset used.

DATA TRAINING



```python
# Create sequences for training
def create_sequences(data, seq_length):
    sequences, labels = [], []
    for i in range(len(data) - seq_length):
        sequences.append(data[i:i + seq_length])
        labels.append(data[i + seq_length])
    return np.array(sequences), np.array(labels)

SEQ_LENGTH = 30
X, y = create_sequences(data_scaled, SEQ_LENGTH)

# Train-test split
split = int(len(X) * 0.8)
X_train, y_train = X[:split], y[:split]
X_test, y_test = X[split:], y[split:]
```

The screenshot above shows the code where the data is trained. It splits the data into training and testing sets. The train-test split played a crucial role for the training of data because it evaluates the model's performance.

BUILD MODEL

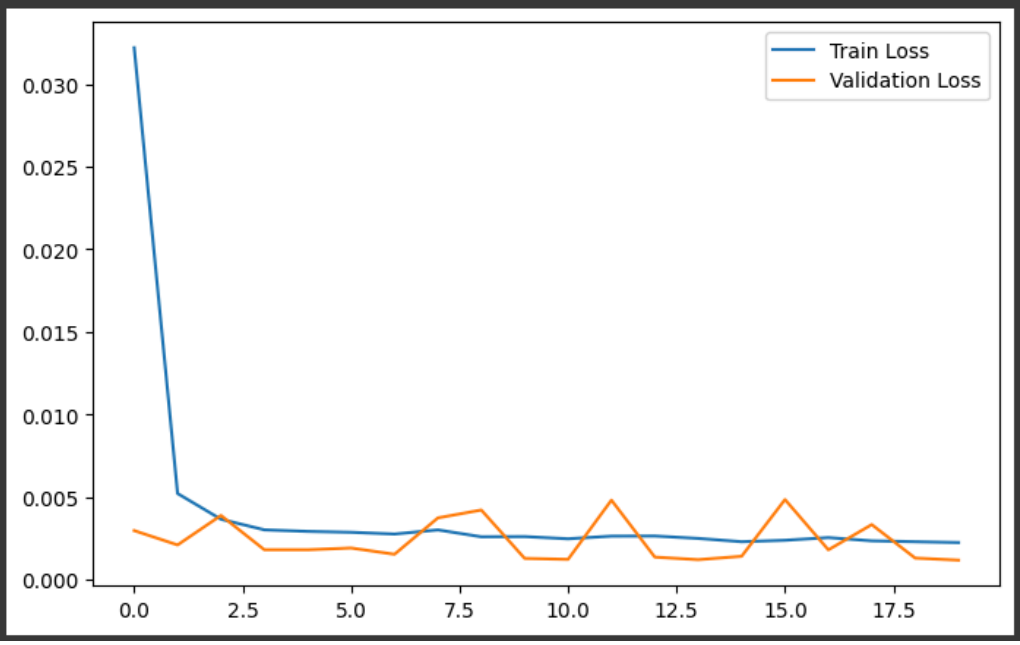
```python
# Build LSTM model
model = Sequential([
    LSTM(50, return_sequences=True, input_shape=(SEQ_LENGTH, 1)),
    Dropout(0.2),
    LSTM(50, return_sequences=False),
    Dropout(0.2),
    Dense(25),
    Dense(1)
])

model.compile(optimizer='adam', loss='mse')

# Train model
history = model.fit(X_train, y_train, epochs=20, batch_size=32, validation_data=(X_test, y_test))
```

```
Epoch 1/20
/usr/local/lib/python3.11/dist-packages/keras/src/layers/rnn/rnn.py:200: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer
  super().__init__(**kwargs)
25/25 ━━━━━━━━━━━━━━━━━━━━ 6s 57ms/step - loss: 0.0684 - val_loss: 0.0030
Epoch 2/20
25/25 ━━━━━━━━━━━━━━━━━━━━ 3s 59ms/step - loss: 0.0058 - val_loss: 0.0021
Epoch 3/20
25/25 ━━━━━━━━━━━━━━━━━━━━ 2s 39ms/step - loss: 0.0038 - val_loss: 0.0039
Epoch 4/20
25/25 ━━━━━━━━━━━━━━━━━━━━ 1s 35ms/step - loss: 0.0032 - val_loss: 0.0018
Epoch 5/20
25/25 ━━━━━━━━━━━━━━━━━━━━ 1s 36ms/step - loss: 0.0030 - val_loss: 0.0018
Epoch 6/20
25/25 ━━━━━━━━━━━━━━━━━━━━ 1s 34ms/step - loss: 0.0026 - val_loss: 0.0019
Epoch 7/20
25/25 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - loss: 0.0027 - val_loss: 0.0015
Epoch 8/20
25/25 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - loss: 0.0031 - val_loss: 0.0037
Epoch 9/20
25/25 ━━━━━━━━━━━━━━━━━━━━ 1s 36ms/step - loss: 0.0026 - val_loss: 0.0042
Epoch 10/20
25/25 ━━━━━━━━━━━━━━━━━━━━ 1s 35ms/step - loss: 0.0024 - val_loss: 0.0013
Epoch 11/20
25/25 ━━━━━━━━━━━━━━━━━━━━ 1s 36ms/step - loss: 0.0024 - val_loss: 0.0012
Epoch 12/20
25/25 ━━━━━━━━━━━━━━━━━━━━ 2s 61ms/step - loss: 0.0026 - val_loss: 0.0048
Epoch 13/20
25/25 ━━━━━━━━━━━━━━━━━━━━ 2s 46ms/step - loss: 0.0027 - val_loss: 0.0014
Epoch 14/20
25/25 ━━━━━━━━━━━━━━━━━━━━ 1s 41ms/step - loss: 0.0024 - val_loss: 0.0012
Epoch 15/20
25/25 ━━━━━━━━━━━━━━━━━━━━ 1s 36ms/step - loss: 0.0022 - val_loss: 0.0014
Epoch 16/20
25/25 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - loss: 0.0026 - val_loss: 0.0049
Epoch 17/20
25/25 ━━━━━━━━━━━━━━━━━━━━ 1s 36ms/step - loss: 0.0024 - val_loss: 0.0018
Epoch 18/20
25/25 ━━━━━━━━━━━━━━━━━━━━ 1s 36ms/step - loss: 0.0026 - val_loss: 0.0033
Epoch 19/20
25/25 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - loss: 0.0024 - val_loss: 0.0013
Epoch 20/20
25/25 ━━━━━━━━━━━━━━━━━━━━ 1s 36ms/step - loss: 0.0022 - val_loss: 0.0012
```

The code above builds and trains a LSTM model for the time series forecasting. It has two LSTM layers with dropout for regularization and a dense output layer. The “model.compile” compiles the model and the “history = model.fit” is used to evaluate the training process.

PLOT OF LOSS

```
# Plot training loss
plt.figure(figsize=(8, 5))
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.show()
```

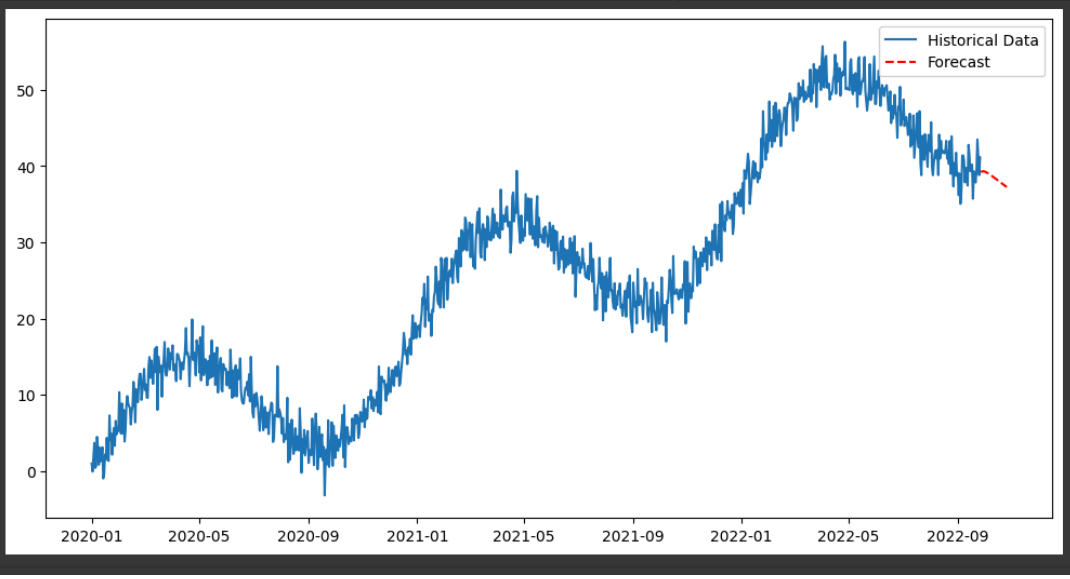


This snippet shows the plot training loss. It visualizes our validation and training loss. This essentially shows us the efficiency of our model because it shows us if it's learning well.

FORECAST PREDICTION

```python
# Forecast next 30 days
def predict_future(model, data, steps):
    predictions = []
    current_input = data[-SEQ_LENGTH:].reshape(1, SEQ_LENGTH, 1)
    for _ in range(steps):
        pred = model.predict(current_input)[0]
        predictions.append(pred)
        current_input = np.append(current_input[:, 1:, :], [[pred]], axis=1)
    return scaler.inverse_transform(predictions)

future_predictions = predict_future(model, data_scaled, 30)
```

```python
# Plot forecast
plt.figure(figsize=(12, 6))
plt.plot(df.index, df['Value'], label='Historical Data')
prediction_dates = pd.date_range(df.index[-1], periods=31, freq='D')[1:]
plt.plot(prediction_dates, future_predictions, label='Forecast', linestyle='dashed', color='red')
plt.legend()
plt.show()
```



The screenshot above shows the forecast prediction. We can see in the visualization that in the years with rising arcs, it is likely to have more rainfalls that will occur in those years. The years with low arcs on the other hand will likely have a low chance for rainfalls to occur. The forecast predicted is specifically in the next 30 days where we can see that in the code, it takes value from the data where it makes a prediction that will be fed back from the model which will repeat 30 times.

REFERENCES

https://www.youtube.com/watch?v=GE3JOFwTWVM&ab_channel=IBMTechnology
https://www.youtube.com/watch?v=vV12dGe_Fho&ab_channel=RobMulla
https://www.youtube.com/watch?v=_mBWIAA4Am4&ab_channel=EgorHowell
https://www.youtube.com/watch?v=j0eioK5edqg&ab_channel=RobMulla
https://www.youtube.com/watch?v=KvLG1uTC-KU&ab_channel=NicholasRenotte